Spark2 SDK — C++ User Manual

C++17 distribution | Windows and Linux

Document version: V1.2

SAFETY — Validate every motion in a clear workspace at a low speed setting. Software limits are an additional safeguard; they do not replace installation checks, emergency-stop procedures, or application-level interlocks.

Document revision history

Version	Date	Notes
V1.0	2026-07-23	Initial separate C++ and Python user manuals.
V1.1	2026-08-06	Document C++17 as the application language level. Restate Windows tools as any Visual Studio version later than 2022 with MSVC toolset v143 (minimum) or newer. Set supported Python versions to 3.10–3.14. Add §2.3 control priority. Clarify Path APIs: at least one of vPath or tPath must match path length. State that t / tPath is in seconds with default 0.
V1.2	2026-08-10	Revise Quick Start for packaged install. Clarify control priority (physical button > standard; same-tier wait). Move defaults/units to §2.4 (v=0 uses max speed). Correct gripper pos as 0–1 mapped to limits. Document limit overrun (bound / fault / jump saturate). Rename state feedback; remove motion-interface selection.

Document scope

This manual is for application developers who use the Spark2 SDK (C++17) to operate one robot arm. It focuses on operational APIs, parameters, defaults, and preconditions; implementation-only helper classes and raw data-structure declarations are intentionally not reproduced.

At a glance

Item	Supported / supplied
Platforms	Windows 10 / 11 (x64); Ubuntu 22.04 (x86 / ARM)
Language	C++17
Build / install	find_package + CMake (C++)
Included examples	Joint point motion and Cartesian point motion, etc.
Document version	V1.2

Contents

- Document revision history
- 1 Installation and first run
- 2 Runtime workflow and conventions
- 3 Core motion API
- 4 Interaction, feedback, and status
- 5 Configuration and safety limits
- 6 Kinematics and frame transforms
- Appendix A API index and type boundary

Terminology

Term	Meaning
Joint space	A command or feedback expressed as six arm-joint values. SDK joint positions use degrees (°); joint velocities use degrees per second (°/s).
EE pose	Pose of the robot end-effector frame.
Tool pose	Pose of the configured tool frame; it includes the active tool offset. Same rotation as EE pose, only translation difference.
Tool offset	Translational offset (Position: x, y, z) of the tool frame relative to the EE frame. Not a full Pose; orientation is not set through the tool-offset API.
PTP	Point-to-point motion. The controller plans a motion to the target pose.
Linear	Cartesian line motion. The commanded frame follows a straight Cartesian path.

1. Installation and first run

1.1 C++ prerequisites

Platform	Required tools	Check
Windows 10 / 11 (x64)	Any Visual Studio version later than 2022 is supported, with the Desktop development with C++ workload and MSVC toolset v143 (minimum) or newer. CMake on PATH.	Open that VS Developer Command Prompt.
Ubuntu 22.04 (x86 / ARM)	A C++17-capable compiler (for example g++ or clang++) and CMake.	Open a terminal in the C++ SDK distribution folder.

1.2 Quick start

1.2.1 Link your application

Consume the installed C++ package with `find_package(spark2_sdk)` and link `spark2::sdk`.

```
cmake_minimum_required(VERSION 3.20)
project(my_app LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)

find_package(spark2_sdk REQUIRED)

add_executable(my_app main.cpp)
target_link_libraries(my_app PRIVATE spark2::sdk)
```

Configure with the install prefix on `CMAKE_PREFIX_PATH`:

```
cmake -S . -B build -DCMAKE_PREFIX_PATH=/path/to/prefix
cmake --build build --parallel
```

On Linux, ensure the loader can see bundled libraries in `<prefix>/lib` (usually automatic via `$ORIGIN` when the executable and SDK are used as packaged). On Windows, copy `spark2_sdk.dll` (and bundled DLLs from `<prefix>/bin`) next to the executable, or add that directory to `PATH`.

1.2.2 Build shipped examples

From an extracted or installed prefix:

```
cmake -S /path/to/prefix/examples/cpp -B build \
  -DCMAKE_PREFIX_PATH=/path/to/prefix
cmake --build build --parallel
```

CHECK — If CMake cannot find a compiler on Windows, reopen the Developer Command Prompt for your installed Visual Studio version. If find_package fails, confirm CMAKE_PREFIX_PATH points at the SDK install prefix. If an executable is built but cannot connect to the robot, check runtime prerequisites (including Windows DLL search path) before changing motion code.

1.3 First-run checklist

1. Confirm the robot, controller, and safety system have completed their normal start-up checks.
2. Use the configuration directory that matches the physical robot and active tool.
3. Keep the arm clear of people, fixtures, cables, and end-of-travel positions.
4. Start at a low motion speed and verify the commanded frame (joint / EE / tool) before increasing speed.
5. Always stop the runtime cleanly when the application exits.

2. Runtime workflow and conventions

2.1 Recommended control sequence

Step	Action	Why it matters
1	Create Spark2 client (Spark2(config_prefix_path = ""))	Loads the SDK client with the intended configuration prefix.
2	start()	Starts the runtime session before motion or feedback calls.
3	Verify status and joint enable state	Avoids issuing a motion command to an unavailable arm.
4	Issue a motion / interaction command	Choose joint, EE, or tool-frame command according to the task.
5	stop()	Ends the runtime session in a controlled manner.

2.2 Client lifecycle

Call	Parameters and result	Use / precondition
Spark2(config_prefix_path = "")	config_prefix_path: optional configuration-file directory. Empty uses the SDK default search location.	Create once before using runtime, configuration, or kinematics services.
start()	No parameters; starts the runtime session.	Call before arm motion, teach, playback, or live feedback.
stop()	No parameters; stops the runtime session.	Call during orderly shutdown; do not rely on process termination as a stop procedure.
enableArmJoint(arm_state)	arm_state: six-element per-joint enable command (true = enable, false = disable).	Use only when the application is responsible for joint enabling and the safety procedure permits it.
setArmSmoothingMethod(method)	method: SDK smoothing strategy for a single-target arm motion.	Select before issuing PTP / line targets when the application needs a specified smoothing behavior.

2.3 Control priority

The real-time controller arbitrates which client may command the robot. Multiple clients can exist at once (for example the physical teach button, Spark2 SDK, web client, and panel). A lower-priority client is blocked while a higher-priority client holds control. Clients in the same priority tier cannot preempt one another: a same-level client must wait until the current controlling client finishes and releases control before it can take control.

Priority (high → low)	Clients (examples)	Implication
Physical button	Physical teach button	Highest among operator clients. When it claims control, standard clients lose exclusive control until it releases.
Standard	Spark2 SDK; Panel; Web	Same priority tier. None of them outranks the others or the physical button. A standard client must wait for the current standard controller to finish and release control before it can take control.

PRIORITY — If control is lost to a higher-priority client (for example the physical button), SDK motion APIs fail with a control-lost error. Restart the program (re-create the client and call start) to reclaim control after that client has released it. If another standard client currently holds control, wait until it finishes and releases control; do not expect to preempt it.

2.4 Defaults and units

Joint position commands and getPos / get_pos feedback use degrees (°). Joint velocity commands (moveVel / move_vel) and getVel / get_vel use degrees per second (°/s). Speed v (and each entry in vPath) is a percentage of

the configured maximum motion speed. Example: v = 50 means about 50% of the active velocity limit; v = 100 means full scale. Default v is 50. If v = 0, the controller uses the maximum speed. Time t (and each entry in tPath) is in seconds (s). Default t is 0. If the time implied by v is longer than the requested t, the controller may extend the actual duration.

3. Core motion API

3.1 Joint-space motion

Call	Parameters	Use / behavior
movePos(arm_pos, v = 50, t = 0)	arm_pos: six-joint target in degrees (°). v: speed % of max (0–100; default 50). t: time in seconds (default 0).	Move to one joint target. Use for a known, collision-checked joint configuration.
movePosPath(path, vPath, tPath)	path: ordered joint waypoints in degrees (°). vPath: per-waypoint speed % (same as v). tPath: per-waypoint times in seconds.	Execute a joint-space waypoint sequence. At least one of vPath or tPath must have the same length as path (both may match; if neither matches, the call fails).
moveVel(arm_vel, a = 10, t = 0)	arm_vel: six-joint velocity in °/s. a: acceleration setting as percentage scale (default 10). t: duration in seconds (default 0).	Command joint velocity. Use bounded duration and an application-level stop condition.
goHome(v = 50, t = 0)	v: speed % of max (0–100; default 50). t: time in seconds (default 0). Home joint pose is all zeros (degrees).	Return to the home pose (all-zero joint angles). Confirm the home path is safe for the installed cell.

MOTION PATH — A successful API call does not by itself prove that the full swept path is safe. Validate fixture clearance, tool geometry, cable routing, and singularity / joint-limit behavior for the complete task path.

3.2 Cartesian motion: EE frame vs. tool frame

Use an EE command when the target is defined at the end-effector frame. Use a tool command when the target is defined at the active tool frame and the configured tool offset should be applied.

Target frame	PTP	Linear	Path variant	When to use
End effector (EE)	moveEEPoint	moveEELine	moveEEPointPath / moveEELinePath	Application defines poses at the end-effector frame.
Tool	moveToolPoint	moveToolLine	moveToolPointPath / moveToolLinePath	Task is specified at the configured tool frame.

3.3 Cartesian parameters

Parameter	Meaning	Practical note
pose	Target Cartesian pose in the selected frame.	Confirm frame convention before commanding motion.
path	Ordered Cartesian waypoints in the selected frame.	Keep frame convention consistent. At least one of vPath or tPath must match path length.
v / vPath	Speed as a percentage (0–100) of the configured maximum motion speed for one target or each path waypoint.	Default v is 50. Keep low during validation. For path APIs, if provided, vPath length must equal path length (or provide a matching tPath instead).
t / tPath	Execution time for one target or each path waypoint, in seconds (s).	Default t is 0. Controller may extend duration if v requires a longer minimum time. For path APIs, if provided, tPath length must equal path length (or provide a matching vPath instead).

FRAME CHECK — Do not mix EE and tool poses. If the tool offset changes, revalidate task poses, approach paths, and any saved waypoint data.

4. Interaction, feedback, and status

4.1 Gripper and teach / playback

Call	Parameters / result	Use / precondition
moveGripperPos(pos, v = 50, t = 0)	pos: target gripper opening as a float in [0, 1] (maps onto the configured gripper limits). v: speed % of max (0–100; default 50). t: time in seconds (default 0).	Command a gripper position. The value is normalized: 0 and 1 correspond to the ends of the active gripper range; intermediate values are mapped into that range.
startTeach() / stopTeach()	No parameters.	Enter or exit hand-guiding mode. Use only with the appropriate physical safety procedure.
startPlayback() / stopPlayback() / resetPlayback()	No parameters.	Start, stop, or reset trajectory playback. Ensure a valid recorded/loaded playback context exists.

4.2 State feedback

Call	Returns / parameters	Use
getPos() / getVel() / getTor()	Six-joint position (°), velocity (°/s), or torque feedback.	Read current joint feedback for monitoring or logging.
getEEPose() / getToolPose()	Current Cartesian pose of the EE or tool frame.	Read the pose in the frame that matches your Cartesian motion commands (EE APIs with EE pose; tool APIs with tool pose).
isArmJointEnabled()	Six-element per-joint enable state.	Confirm joints are enabled before commanding motion.
getStatus()	Aggregated runtime status (SystemStatus).	Inspect robot state, plan result, and diagnostic flags.
printStatus(status)	status: SystemStatus from getStatus / get_status.	Print status fields for debugging or commissioning.

SUPERVISION — For production applications, poll the appropriate status source in your control loop and define a safe action for communication loss, disabled joints, following-error conditions, or an application timeout.

5. Configuration and safety limits

Obtain the configuration service through getConfigurator(). Limit APIs are software safeguards used by the SDK. They must be reviewed together with physical stops, the robot's configured hardware limits, and the system-level safety design.

Call	Parameters / result	Use / caution
getConfigurator()	Returns the Configurator service.	Use once the client is created; retain per binding lifetime rules.
setToolOffset(Position) / getToolOffset()	offset: Position (x, y, z) translational tool offset relative to EE — not a full Pose.	Set and verify before any tool-frame motion. Changing it changes the meaning of tool targets.
setArmPosLimits(limit)	limit: six-joint position software limit.	Set only after checking hardware capability and task envelope. Commands beyond this limit are bounded at the limit.
setArmVelLimits(limit)	limit: six-joint velocity software limit.	Set only after checking hardware capability and task envelope.
getArmPosLimits() / getArmVelLimits()	Returns active software limits.	Read back during commissioning and before task execution.
getArmMaxPos()	Hardware maximum position constraints.	Exceeding this limit puts the robot into fault mode; recovery is required.
getArmMaxVel()	Hardware maximum velocity constraints.	Reference ceiling; not a recommended operating value.
getArmMaxPosJump() / getArmMaxVelJump()	Anti-jump protection thresholds.	Avoid overly large jumps between successive targets. If an interpolated step exceeds this threshold, the command is saturated at the jump limit.
getArmMaxPosFollowError()	Position following-error threshold.	Exceeding this limit puts the robot into fault mode; recovery is required. This typically appears only when a motor or drive has a problem.
getArmMaxVelFollowError() / getArmMaxTorFollowError()	Following-error thresholds for velocity and torque.	Use for monitoring expectations and recovery logic.
setGripperPosLimits(limit) / getGripperPosLimits() / getGripperMaxPos()	Gripper software limits and hardware maximum.	Defines the physical range that moveGripperPos / move_gripper_pos maps from the normalized pos in [0, 1].

CONFIG — Keep configuration files and tool offsets under version control for each robot serial number and end-effector. Record the active configuration with test results whenever a limit or offset is changed.

6. Kinematics and frame transforms

Obtain the kinematics service through getKinematics(). These utilities support planning, target validation, frame conversion, and offline calculation; they do not start motion by themselves.

Call	Input / output	Use
getKinematics()	Returns the Kinematics service.	Access kinematic calculations for the active SDK configuration.
forwardKinematics(arm_pos)	Input: six-joint arm state in degrees (°). Output: EE Cartesian pose.	Compute EE pose for a known joint state.
inverseKinematics(ee_pose, arm_pos)	Input: EE target pose. Output: success flag and solved six-joint state in degrees (°).	Check the Boolean success result before using the solved joints.
eeToToolPose(ee_pose, tool_offset)	Converts an EE pose to a tool pose using a translational tool offset (Position).	Explicit frame conversion in planning or logging.
toolToEEPose(tool_pose, tool_offset)	Converts a tool pose to an EE pose using a translational tool offset (Position).	When a tool-defined target must be expressed at the EE frame.

6.1 Orientation conversion utilities

Call	Use
eulerToQuaternion(euler)	Convert intrinsic XYZ Euler angles to a quaternion orientation.
quaternionToEuler(quat)	Convert a quaternion orientation to intrinsic XYZ Euler angles.

IK — An inverse-kinematics solution is only a kinematic result. Validate joint limits, collisions, singularity margin, and the desired approach path before passing it to a motion command.

Appendix A. API index and type boundary

A.1 API groups

Group	Public operations in this manual
Lifecycle	Spark2, start, stop, enableArmJoint, setArmSmoothingMethod
Joint motion	movePos, movePosPath, moveVel, goHome
Cartesian motion	moveEEPoint, moveEEPointPath, moveEELine, moveEELinePath, moveToolPoint, moveToolPointPath, moveToolLine, moveToolLinePath
Interaction and feedback	moveGripperPos, startTeach, stopTeach, startPlayback, stopPlayback, resetPlayback, getPos, getVel, getTor, getEEPose, getToolPose, isArmJointEnabled, getStatus, printStatus
Configuration	getConfigurator, set/get tool offset, arm limits, gripper limits, maximums, anti-jump and following-error thresholds
Kinematics	getKinematics, forwardKinematics, inverseKinematics, Euler/quaternion conversion, EE/tool transforms

A.2 Types intentionally kept concise

The SDK headers and language bindings are the authoritative source for exact memory layout, constructors, enum values, and serialization behavior. This user manual only uses the following operational interpretation:

Type family	Operational meaning in this manual
JointState6f / JointState6b	Six-element arm joint values (positions in degrees for motion/FK/IK APIs) or six-element joint enable states.
Path<T>	Ordered list of targets or waypoints of type T.
Pose / Quaternion / EulerAngle	SDK geometric types. Confirm coordinate convention before integration.
Position	Translational XYZ type used for tool offset (setToolOffset / set_tool_offset), not orientation.
JointLimits6f / JointLimits1f	Software limits for the arm or gripper.
SystemStatus / SmoothingMethod	Runtime status and motion-smoothing SDK types.

REF — This edition deliberately omits internal helper declarations. Use the distributed C++ headers in spark2_cpp_dist when implementing a type-specific integration.